LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH COMPOSE

Oleh:

Ahmad Luthfi Maulana NIM. 2410817310006

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Ahmad Luthfi Maulana
NIM : 2410817310006

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Raymond Hariyono	Erika Maulidiya, S.Kom., M.Kom
NIM. 2310817210007	NIP. 200006192025062016

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 8
Source Code XML 8
Source Code Compose 12
B. Output Program 16
C. Pembahasan 17
Pembahasan XML 17
Pembahasan Compose 19
SOAL 2 21
SOAL 3 23
Daftar Pustaka 25
Tautan Git 26

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 XML..... 16
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose 17

DAFTAR TABEL

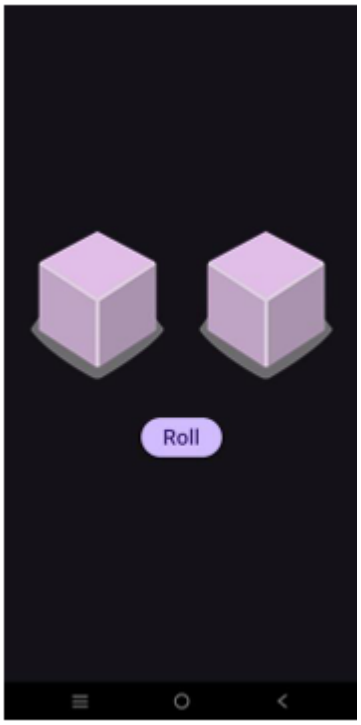
Tabel 1. Source Code Jawaban Soal 1 XML 8
Tabel 2. Source Code Jawaban Soal 1 XML 10
Tabel 3. Source Code Jawaban Soal 1 Compose 12

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.

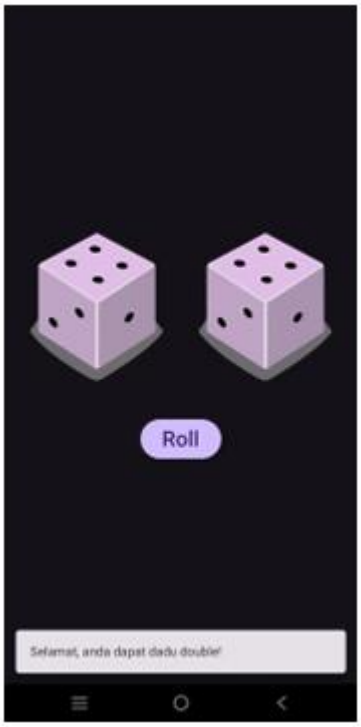


Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.

Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.



Gambar 3 Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam folder Modul 1 dalam bentuk Project. Jangan lupa untuk melakukan Clean Project sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

Source Code XML
MainActivity.kt

Tabel 1. Source Code Jawaban Soal 1 XML

```
package com.example.diceroller

import android.os.Bundle
import android.widget.Button
```

```
import android.widget.ImageView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    private lateinit var diceImage1: ImageView
    private lateinit var diceImage2: ImageView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rollButton: Button = findViewById(R.id.button)
        diceImage1 = findViewById(R.id.dice1)
        diceImage2 = findViewById(R.id.dice2)

        diceImage1.setImageResource(R.drawable.empty_dice)
        diceImage2.setImageResource(R.drawable.empty_dice)

        rollButton.setOnClickListener {
            rollDice()
        }
    }

    private fun rollDice() {
        val diceRoll1 = (1..6).random()
        val diceRoll2 = (1..6).random()


diceImage1.setImageResource(getDiceResource(diceRoll1))

diceImage2.setImageResource(getDiceResource(diceRoll2))

        if (diceRoll1 == diceRoll2) {
```

```
            Toast.makeText(this, "Selamat, anda dapat dadu double!", Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "Anda belum beruntung!", Toast.LENGTH_SHORT).show()
        }
    }

    private fun getDiceResource(roll: Int): Int {
        return when (roll) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
    }
}
```

activity_main.xml

Tabel 2. Source Code Jawaban Soal 1 XML

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#121212">

    <ImageView
        android:id="@+id/dice1"
        android:layout_width="150dp"
```

```
        android:layout_height="150dp"
        android:contentDescription="@string/dice_1_desc"
        android:src="@drawable/empty_dice"
        app:layout_constraintBottom_toTopOf="@+id/button"
        app:layout_constraintEnd_toStartOf="@+id/dice2"
        app:layout_constraintHorizontal_chainStyle="packed"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.5"           />

    <ImageView
        android:id="@+id/dice2"
        android:layout_width="150dp"
        android:layout_height="150dp"
        android:layout_marginStart="16dp"
        android:contentDescription="@string/dice_2_desc"
        android:src="@drawable/empty_dice"
        app:layout_constraintBottom_toBottomOf="@+id/dice1"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/dice1"
        app:layout_constraintTop_toTopOf="@+id/dice1"      />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:backgroundTint="#D1C4E9"
        android:text="@string/roll_text"
        android:textColor="#311B92"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/dice1"
```

```
        app:layout_constraintVertical_bias="0.1"            />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Source Code Compose

MainAcivity.kt

Tabel 3. Source Code Jawaban Soal 1 Compose

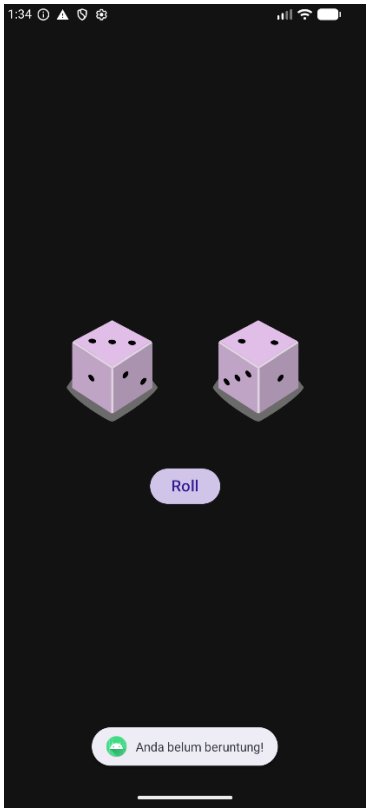
```
package com.example.dicerollercompose

import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
```

```
27                DiceRollerApp()
28            }
29        }
30    }
31
32    @Composable
33    fun DiceRollerApp() {
34        val context = LocalContext.current
35
36        var diceResult1 by remember { mutableIntStateOf(0) }
37        var diceResult2 by remember { mutableIntStateOf(0) }
38
39        val imageResource1 = when (diceResult1) {
40            1 -> R.drawable.dice_1
41            2 -> R.drawable.dice_2
42            3 -> R.drawable.dice_3
43            4 -> R.drawable.dice_4
44            5 -> R.drawable.dice_5
45            6 -> R.drawable.dice_6
46            else -> R.drawable.empty_dice
47        }
48
49        val imageResource2 = when (diceResult2) {
50            1 -> R.drawable.dice_1
51            2 -> R.drawable.dice_2
52            3 -> R.drawable.dice_3
53            4 -> R.drawable.dice_4
54            5 -> R.drawable.dice_5
55            6 -> R.drawable.dice_6
56            else -> R.drawable.empty_dice
57        }
58
59        Surface(
60            modifier = Modifier.fillMaxSize(),
```

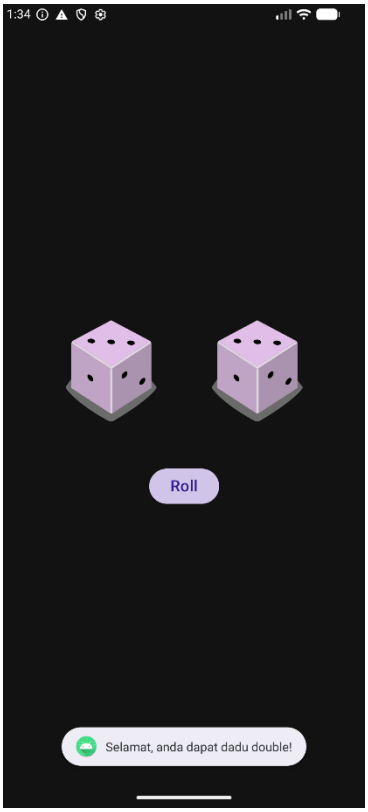
```
        color = Color(0xFF121212)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                Image(
                    painter = painterResource(imageResource1),
                    contentDescription = null,
                    modifier = Modifier.size(150.dp)
                )
                Image(
                    painter = painterResource(imageResource2),
                    contentDescription = null,
                    modifier = Modifier.size(150.dp)
                )
            }

            Spacer(modifier = Modifier.height(32.dp))

            Button(
                onClick = {
                    diceResult1 = (1..6).random()
                    diceResult2 = (1..6).random()
```

```

                    val message = if (diceResult1 == diceResult2) {
                        "Selamat, anda dapat dadu double!"
                    } else {
                        "Anda belum beruntung!"
                    }
                    Toast.makeText(context, message, Toast.LENGTH_SHORT).show()
                },
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color(0xFFD1C4E9),
                    contentColor = Color(0xFF311B92)
                ),
                shape = RoundedCornerShape(50.dp)
            ) {
                Text(text = "Roll", fontSize = 18.sp)
            }
        }
    }
}
```

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1 XML



Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

Pembahasan XML

MainActivity.kt:

Line 1: Mendeklarasikan paket file Kotlin yaitu com.example.diceroller.

Line 3 - 7: Mengimpor kelas-kelas yang diperlukan seperti Bundle untuk *lifecycle*, serta Button, ImageView, dan Toast untuk komponen UI.

Line 9: Mendeklarasikan kelas MainActivity yang merupakan turunan dari AppCompatActivity.

Line 10 - 11: Mendeklarasikan variabel diceImage1 dan diceImage2 menggunakan lateinit agar dapat diinisialisasi nanti.

Line 13 - 15: Fungsi onCreate dipanggil saat aktivitas dibuat untuk mengatur layout activity_main.xml.

Line 17 - 19: Menghubungkan variabel dengan elemen UI di XML menggunakan fungsi findViewById().

Line 21 - 22: Mengatur gambar awal kedua dadu menggunakan aset empty_dice dari folder *drawable*.

Line 24 - 26: Memberikan *listener* klik pada rollButton untuk menjalankan fungsi rollDice().

Line 29 - 31: Fungsi rollDice() menghasilkan angka acak 1 sampai 6 untuk kedua dadu menggunakan random().

Line 33 - 34: Memperbarui tampilan gambar dadu di layar berdasarkan angka acak yang didapat melalui fungsi getDiceResource().

Line 36 - 40: Menggunakan pengkondisian if-else untuk menampilkan pesan Toast "Selamat, anda dapat dadu double!" jika angka sama, atau "Anda belum beruntung!" jika berbeda.

Line 43 - 51: Fungsi getDiceResource() menggunakan ekspresi when untuk memetakan angka hasil kocokan ke aset gambar dadu yang sesuai di *drawable*.

activity_main.xml:

Line 1: Mendeklarasikan file sebagai dokumen XML dengan encoding UTF-8.

Line 2 - 6: Membuka ConstraintLayout sebagai tata letak utama aplikasi dengan lebar dan tinggi memenuhi seluruh layar (*match_parent*).

Line 7: Mengatur warna latar belakang menjadi hitam gelap (#121212).

Line 9: Membuat komponen ImageView dengan ID @+id/dice1 untuk menampilkan dadu pertama.

Line 10 - 11: Menentukan ukuran gambar dadu sebesar 150dp x 150dp.

Line 13: Menampilkan gambar awal dadu kosong menggunakan @drawable/empty_dice.

Line 14 - 19: Mengatur posisi dadu pertama agar berada di tengah layar secara horizontal dan vertikal menggunakan sistem *chain*.

Line 21: Membuat komponen ImageView dengan ID @+id/dice2 untuk menampilkan dadu kedua.

Line 23: Memberikan jarak (*margin*) sebesar 16dp di sebelah kiri dadu kedua agar tidak menempel dengan dadu pertama.

Line 26 - 29: Menghubungkan posisi dadu kedua agar sejajar secara vertikal dengan dadu pertama.

Line 31: Membuat komponen Button dengan ID @+id/button untuk memicu aksi pengocokan dadu.

Line 32 - 33: Lebar tombol menyesuaikan isi teks (*wrap_content*) dengan tambahan jarak atas (*margin top*) sebesar 32dp.

Line 35 - 37: Mengatur warna latar belakang tombol menjadi ungu muda (#D1C4E9) dan warna teks menjadi ungu tua (#311B92).

Line 39 - 42: Menempatkan posisi tombol tepat di bawah kedua gambar dadu dan berada di tengah secara horizontal.

Line 45: Menutup tag ConstraintLayout.

Pembahasan Compose

MainActivity.kt

Line 1: Mendeklarasikan paket file Kotlin yaitu com.example.dicerollercompose.

Line 3 - 22: Merupakan *import statement* untuk memuat library Android, komponen Jetpack Compose, dan UI Material3 yang diperlukan.

Line 24: Mendeklarasikan kelas MainActivity yang merupakan turunan dari ComponentActivity, superclass standar untuk aktivitas berbasis Compose.

Line 25 - 30: Fungsi onCreate() menggunakan setContent untuk menetapkan fungsi komposable DiceRollerApp() sebagai konten UI utama.

Line 34: Menandai fungsi DiceRollerApp() dengan anotasi @Composable agar dapat digunakan untuk menghasilkan antarmuka pengguna.

Line 35: Menggunakan LocalContext.current untuk mendapatkan konteks aplikasi yang diperlukan untuk menampilkan *Toast*.

Line 37 - 38: Mendeklarasikan variabel *state* diceResult1 dan diceResult2 menggunakan remember agar nilai angka dadu tetap tersimpan selama komposisi ulang.

Line 40 - 58: Menggunakan ekspresi when untuk menentukan *resource* gambar dadu (imageResource1 dan imageResource2) berdasarkan angka hasil kocokan.

Line 60 - 63: Komponen Surface digunakan sebagai wadah utama layar dengan warna latar belakang gelap (0xFF121212).

Line 64 - 68: Komponen Column mengatur tata letak elemen secara vertikal, berada di tengah layar dengan Alignment.CenterHorizontally.

Line 69 - 72: Komponen Row menempatkan dua gambar dadu secara horizontal dengan jarak antar elemen sebesar 16.dp.

Line 73 - 82: Komponen Image memuat gambar dadu sesuai hasil kocokan menggunakan painterResource dengan ukuran masing-masing 150.dp.

Line 85: Spacer memberikan jarak kosong setinggi 32.dp antara gambar dadu dan tombol.

Line 87 - 100: Membuat Button dengan logika klik untuk mengacak angka dadu menggunakan (1..6).random() serta menentukan pesan *Toast* berdasarkan hasil angka yang didapat.

Line 101 - 104: Mengatur warna tombol menjadi ungu muda (containerColor) dan warna teks tombol menjadi ungu tua (contentColor).

Line 105: Memberikan bentuk sudut tombol yang membulat penuh menggunakan RoundedCornerShape(50.dp).

Line 107: Menampilkan teks "Roll" di dalam tombol dengan ukuran font 18.sp.

SOAL 2

Riset dan cite minimal 2 buah artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.

1. Imperative Programming

Pemrograman imperatif adalah paradigma yang menjelaskan instruksi secara detail mengenai setiap langkah yang harus diambil oleh komputer untuk mencapai tujuan tertentu. Dalam konteks pembuatan UI, pengembang harus mengelola perubahan *state* secara manual, seperti mendapatkan referensi komponen melalui findViewById() dan memperbarui propertinya secara eksplisit satu per satu (Jemerov & Isakova, 2017). Digunakan pada pengembangan Android tradisional menggunakan XML (*Android Views*), di mana pengembang menulis kode untuk memanipulasi hirarki *view* saat terjadi interaksi pengguna.

2. Declarative Programming

Pemrograman deklaratif adalah paradigma yang berfokus pada hasil akhir atau "apa" yang ingin ditampilkan, tanpa merinci langkah-langkah prosedural untuk mengubah *state* (Abelson & Sussman, 1996). Dalam UI, pengembang cukup mendeskripsikan tampilan untuk setiap kondisi *state* yang mungkin terjadi, dan kerangka kerja (*framework*) akan menangani proses pembaruan tampilannya secara otomatis (Griffiths & Griffiths, 2019). Diimplementasikan pada Jetpack Compose, di mana UI didefinisikan sebagai fungsi dari data (*state*). Ketika data berubah, fungsi komposable akan dipanggil ulang untuk memperbarui tampilan.

Opini dan Preferensi Paradigma

Berdasarkan riset dan pengalaman selama mengerjakan Modul 1 ini, saya lebih memilih Paradigma Declarative (Jetpack Compose). Alasan yang mendasari preferensi tersebut:

1. **Efisiensi Kode**: Kode yang dihasilkan jauh lebih ringkas dibandingkan versi imperatif karena tidak memerlukan sinkronisasi manual antara file logic (Kotlin) dan file layout (XML).

2. **Keamanan State**: Dalam paradigma deklaratif, UI selalu mencerminkan data terbaru secara otomatis melalui *state management* (remember, mutableStateOf), sehingga meminimalisir risiko kesalahan sinkronisasi data yang sering terjadi pada model imperatif.
3. **Kemudahan Pemeliharaan**: Karena logika dan deskripsi UI berada dalam satu bahasa (Kotlin), proses pencarian kesalahan (*debugging*) menjadi lebih mudah karena tidak perlu berpindah-pindah antar file yang berbeda format.

SOAL 3

Apa yang membedakan OOP pada kotlin dengan OOP pada java?

Meskipun Kotlin dan Java sama-sama berjalan di atas *Java Virtual Machine* (JVM) dan mendukung paradigma Pemrograman Berorientasi Objek (OOP), Kotlin memperkenalkan beberapa fitur modern untuk menyempurnakan kekurangan desain pada Java (Jemerov & Isakova, 2017). Berikut adalah yang membedakan:

1. Null Safety Di Java, semua objek secara default dapat bernilai null, yang sering menyebabkan *NullPointerException* (NPE) saat aplikasi berjalan. Kotlin memiliki sistem tipe yang membedakan antara tipe yang boleh null (*nullable*) dan tidak boleh null (*non-nullable*) secara eksplisit (Leiva, 2017). Hal ini memberikan perlindungan pada tingkat kompilasi agar aplikasi lebih stabil.

2. Properties vs Fields Java menggunakan *fields* untuk menyimpan data dan memerlukan pembuatan fungsi *getter* serta *setter* secara manual (boilerplate code). Kotlin menggunakan konsep Property, di mana *getter* dan *setter* secara otomatis dihasilkan oleh kompiler, sehingga kode menjadi jauh lebih ringkas dan bersih (Jemerov & Isakova, 2017).

3. Class Inheritance (Open by Default vs Final by Default) Pada Java, semua kelas bersifat *open* (dapat diwarisi) kecuali ditandai secara eksplisit dengan kata kunci final. Sebaliknya, di Kotlin semua kelas bersifat final secara default (Jemerov & Isakova, 2017). Jika ingin sebuah kelas dapat diwarisi oleh kelas lain, pengembang harus menambahkan kata kunci open. Hal ini mendorong praktik desain perangkat lunak yang lebih aman.

4. Data Classes Kotlin menyediakan fitur data class yang secara otomatis menghasilkan fungsi-fungsi standar seperti equals(), hashCode(), dan toString() (Leiva, 2017). Di Java, pengembang harus menuliskan kode-kode tersebut secara manual untuk setiap kelas model data, yang seringkali memakan waktu dan rentan kesalahan.

5. Extension Functions Salah satu fitur OOP paling kuat di Kotlin yang tidak dimiliki Java standar adalah Extension Functions. Fitur ini memungkinkan pengembang untuk menambah

fungsi baru ke dalam kelas yang sudah ada tanpa harus melakukan pewarisan (*inheritance*) atau mengubah kode sumber asli dari kelas tersebut (Leiva, 2017).

Daftar Pustaka

Abelson, H., & Sussman, G. J. (1996). Structure and Interpretation of Computer Programs. *MIT Press Journals*, 1-20.

Griffiths, D., & Griffiths, D. (2019). *Head First Android Development: A Learner's Guide to Building Android Apps with Kotlin.* Sebastopol: O'Reilly Media.

Jemerov, D., & Isakova, S. (2017). *Kotlin in Action.* Shelter Island: Manning Publications.

Leiva, A. (2017). *Kotlin for Android Developers: Learn Kotlin the easy way while developing an Android App.* Leanpub.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/anomaliemas/Praktikum-Mobile